



Mountain Subarray Queries

Problem

Given an array `arr[]` and multiple queries:

```
[l, r]
```

For each query, determine whether the subarray:

```
arr[l...r]
```

is a **mountain array**.

A subarray is a mountain if there exists an index `k` such that:

```
l <= k <= r
```

and:

```
arr[l] <= arr[l + 1] <= ... <= arr[k]
```

then:

```
arr[k] >= arr[k + 1] >= ... >= arr[r]
```

So the pattern is:

```
NON-DECREASING → NON-INCREASING
```

1. Understand the Mountain Shape

Valid Mountain

```
[2, 3, 4, 5, 4, 3, 2]
  ↑  ↑  ↑  ↓  ↓  ↓
          PEAK
```

Actually, visually:

```
2 → 3 → 4 → 5 → 4 → 3 → 2
              ↑
            PEAK
```

The array has:

```
Increasing part:
2 <= 3 <= 4 <= 5

Decreasing part:
5 >= 4 >= 3 >= 2
```

Therefore:

Valid Mountain

2. Entirely Increasing Is Also Valid

```
[1, 2, 3, 4]
```

```
1 <= 2 <= 3 <= 4
```

This is valid.

The peak can be the last element:

```
1 → 2 → 3 → 4
          PEAK
```

3. Entirely Decreasing Is Also Valid

```
[5, 4, 3, 2]
```

```
5 >= 4 >= 3 >= 2
```

This is valid.

The peak can be the first element:

```
PEAK  
↓  
5 → 4 → 3 → 2
```

4. Equal Elements Are Allowed

The conditions are:

```
<=  
>=
```

not:

```
<  
>
```

Therefore:

```
[2, 2, 2, 2]
```

is valid.

Because:

```
2 <= 2 <= 2 <= 2
```

and:

2 >= 2 >= 2 >= 2

5. The Invalid Pattern

A mountain can have:

UP → DOWN

But it cannot have:

DOWN → UP

Example:

[3, 2, 4]
↓ ↑

This is invalid.

Why?

Because:

3 → 2

is decreasing, but then:

2 → 4

increases again.

So the array has:

DECREASE → INCREASE

which is not a mountain.

6. Brute-Force Thinking

For every query:

```
[1, r]
```

we can try every possible peak.

Example:

```
arr = [2, 3, 4, 3, 2]
      0  1  2  3  4
```

Query:

```
[0, 4]
```

Possible peaks:

```
k = 0
k = 1
k = 2
k = 3
k = 4
```

Try:

```
k = 2
```

Check the left side:

```
2 <= 3 <= 4
```

Check the right side:

```
4 >= 3 >= 2
```

Therefore:

```
true
```

But this can be slow.

If:

```
n = 100000  
q = 100000
```

checking every element for every query can approach:

```
O(n × q)
```

We want:

```
Preprocessing: O(n)  
  
Each query: O(1)  
  
Total: O(n + q)
```

7. Core Observation

For every query:

```
[1, r]
```

we need to find a peak:

```
k
```

such that:

$$l \leq k \leq r$$

and:

$$l \rightarrow k$$

is non-decreasing:

$$\text{arr}[l] \leq \text{arr}[l+1] \leq \dots \leq \text{arr}[k]$$

and:

$$k \rightarrow r$$

is non-increasing:

$$\text{arr}[k] \geq \text{arr}[k+1] \geq \dots \geq \text{arr}[r]$$

Visual:

l k r
 $|$ ----- $|$ ----- $|$
 NON-DECREASING NON-INCREASING

Therefore, the real problem is:

For every index, know how far the increasing and decreasing sequences can extend.

This leads to preprocessing arrays.

8. The `inc[]` Array

We define:

`inc[i]` = the farthest index reachable starting from `i`
while the array remains non-decreasing

In simple words:

If I start at index i , how far right can I go while values keep increasing or staying equal?

Example:

```
arr = [2, 3, 4, 5, 4, 3, 2]
      0 1 2 3 4 5 6
```

Starting at index 0:

$2 \leq 3 \leq 4 \leq 5$

But:

$5 \leq 4$ ❌

Therefore:

$inc[0] = 3$

Starting at index 1:

$3 \leq 4 \leq 5$

So:

$inc[1] = 3$

Starting at index 2:

$4 \leq 5$

So:

$inc[2] = 3$

Starting at index 3:

5 <= 4 ✖

So:

inc[3] = 3

Starting at index 4:

4 <= 3 ✖

So:

inc[4] = 4

Starting at index 5:

3 <= 2 ✖

So:

inc[5] = 5

Index 6 has no next element:

inc[6] = 6

Final:

inc = [3, 3, 3, 3, 4, 5, 6]

9. Important Question

Why do we calculate `inc[]` from right to left?

At first, it feels like we should do:

```
left → right
```

because we are asking:

```
Starting from i, how far can I go to the RIGHT?
```

That thinking is correct.

However, our recurrence is:

```
inc[i] = inc[i + 1]
```

So:

```
inc[i]
```

depends on:

```
inc[i + 1]
```

Therefore, `inc[i + 1]` must already be calculated.

That means:

```
i + 1
```

must be processed before:

```
i
```

Therefore:

```
RIGHT → LEFT
```

10. Deriving the `inc[]` Recurrence

Suppose:

```
arr[i] <= arr[i + 1]
```

Example:

```
arr[i]      = 3  
arr[i + 1] = 4
```

If index `i` can move to index `i + 1`, then:

```
inc[i] = inc[i + 1]
```

Example:

```
arr = [2, 3, 4, 5]  
      0  1  2  3
```

We already know:

```
inc[1] = 3
```

because:

```
3 <= 4 <= 5
```

Now:

```
arr[0] <= arr[1]  
2 <= 3
```

Therefore:

```
inc[0] = inc[1]
```

So:

```
inc[0] = 3
```

Because:

```
2 <= 3 <= 4 <= 5
```

If the condition fails

Suppose:

```
arr[i] > arr[i + 1]
```

Example:

```
5 > 4
```

Then the non-decreasing sequence stops immediately.

Therefore:

```
inc[i] = i
```

So the recurrence is:

```
if arr[i] <= arr[i + 1]:  
    inc[i] = inc[i + 1]  
else:  
    inc[i] = i
```

11. Full `inc[]` Code

```
inc = [0] * n

inc[n - 1] = n - 1

for i in range(n - 2, -1, -1):

    if arr[i] <= arr[i + 1]:
        inc[i] = inc[i + 1]

    else:
        inc[i] = i
```

12. `inc[]` Dry Run

Consider:

```
arr = [2, 3, 4, 5, 4, 3, 2]
      0  1  2  3  4  5  6
```

Initially:

```
inc = [0, 0, 0, 0, 0, 0, 0]
```

Set the last index:

```
inc[6] = 6
```

Why?

Because there is no element after index 6.

```
inc = [0, 0, 0, 0, 0, 0, 6]
```

`i = 5`

```
arr[5] = 3
arr[6] = 2
```

Check:

$3 \leq 2$ ❌

Therefore:

$\text{inc}[5] = 5$

$\text{inc} = [0, 0, 0, 0, 0, 5, 6]$

$i = 4$

$\text{arr}[4] = 4$
 $\text{arr}[5] = 3$

$4 \leq 3$ ❌

Therefore:

$\text{inc}[4] = 4$

$\text{inc} = [0, 0, 0, 0, 4, 5, 6]$

$i = 3$

$\text{arr}[3] = 5$
 $\text{arr}[4] = 4$

$5 \leq 4$ ❌

Therefore:

```
inc[3] = 3
```

```
inc = [0, 0, 0, 3, 4, 5, 6]
```

i = 2

```
arr[2] = 4  
arr[3] = 5
```

```
4 <= 5 
```

Therefore:

```
inc[2] = inc[3]
```

```
inc[2] = 3
```

```
inc = [0, 0, 3, 3, 4, 5, 6]
```

i = 1

```
arr[1] = 3  
arr[2] = 4
```

```
3 <= 4 
```

Therefore:

```
inc[1] = inc[2]
```

```
inc[1] = 3
```

```
inc = [0, 3, 3, 3, 4, 5, 6]
```

i = 0

```
arr[0] = 2  
arr[1] = 3
```

```
2 <= 3 
```

Therefore:

```
inc[0] = inc[1]
```

```
inc[0] = 3
```

Final:

```
inc = [3, 3, 3, 3, 4, 5, 6]
```

13. Understanding the **inc[]** Output

```
inc = [3, 3, 3, 3, 4, 5, 6]
```

Interpretation:

```
inc[0] = 3
```

means:

Starting at index 0,
we can go until index 3
while maintaining non-decreasing order.

That subarray is:

```
arr[0...3] = [2, 3, 4, 5]
```

```
inc[1] = 3
```

means:

```
arr[1...3] = [3, 4, 5]
```

is non-decreasing.

```
inc[4] = 4
```

means:

Starting from index 4,
we cannot move to index 5
because:

$4 \leq 3$ ❌

So the maximum reachable index is itself:

4

14. The `dec[]` Array

Now we need the other half.

We define:

```
dec[i] = the farthest index reachable while moving LEFT  
         in non-increasing order
```

Another way to say it:

If I want to reach index i through a non-increasing sequence, how far left can that sequence start?

Example:

```
arr = [2, 3, 4, 5, 4, 3, 2]  
       0  1  2  3  4  5  6
```

The decreasing sequence is:

```
5 >= 4 >= 3 >= 2
```

It starts at index 3.

Therefore:

```
dec[6] = 3
```

Similarly:

```
dec[5] = 3  
dec[4] = 3  
dec[3] = 3
```

The final array is:

```
dec = [0, 1, 2, 3, 3, 3, 3]
```

15. Deriving the $dec[]$ Recurrence

We scan from left to right.

Suppose:

```
arr[i - 1] >= arr[i]
```

Example:

```
5 >= 4
```

Then the non-increasing sequence continues.

So:

```
dec[i] = dec[i - 1]
```

Example:

```
arr = [5, 4, 3, 2]
      0  1  2  3
```

We know:

```
dec[1] = 0
```

because:

```
5 >= 4
```

Now:

```
4 >= 3
```

So:

```
dec[2] = dec[1]
```

Therefore:

```
dec[2] = 0
```

The sequence still starts at index 0.

If the condition fails

Suppose:

```
arr[i - 1] < arr[i]
```

Example:

```
3 < 4
```

Then the decreasing sequence cannot continue from the previous index.

Therefore:

```
dec[i] = i
```

So:

```
if arr[i - 1] >= arr[i]:
    dec[i] = dec[i - 1]
else:
    dec[i] = i
```

16. Full `dec[]` Code

```
dec = [0] * n

dec[0] = 0

for i in range(1, n):

    if arr[i - 1] >= arr[i]:
        dec[i] = dec[i - 1]
```

```
else:  
    dec[i] = i
```

17. `dec[]` Dry Run

```
arr = [2, 3, 4, 5, 4, 3, 2]  
      0  1  2  3  4  5  6
```

Initially:

```
dec[0] = 0
```

`i = 1`

```
arr[0] = 2  
arr[1] = 3
```

Check:

```
2 >= 3 ❌
```

So:

```
dec[1] = 1
```

`i = 2`

```
arr[1] = 3  
arr[2] = 4
```

```
3 >= 4 ❌
```

So:

```
dec[2] = 2
```

i = 3

```
arr[2] = 4  
arr[3] = 5
```

```
4 >= 5 ❌
```

So:

```
dec[3] = 3
```

i = 4

```
arr[3] = 5  
arr[4] = 4
```

```
5 >= 4 ✅
```

Therefore:

```
dec[4] = dec[3]
```

```
dec[4] = 3
```

i = 5

```
arr[4] = 4  
arr[5] = 3
```

4 >= 3 

Therefore:

```
dec[5] = dec[4]
```

```
dec[5] = 3
```

i = 6

```
arr[5] = 3  
arr[6] = 2
```

3 >= 2 

Therefore:

```
dec[6] = dec[5]
```

```
dec[6] = 3
```

Final:

```
dec = [0, 1, 2, 3, 3, 3, 3]
```

18. Understanding the **dec[]** Output

```
dec[6] = 3
```

means:

The longest non-increasing sequence ending at index 6 starts at index 3.

That sequence is:

```
arr[3...6] = [5, 4, 3, 2]
```

```
dec[5] = 3
```

means:

```
arr[3...5] = [5, 4, 3]
```

is non-increasing.

19. Query Logic

For a query:

```
[1, r]
```

we need to find whether an increasing section from **1** and a decreasing section ending at **r** can meet.

The increasing section from **1** can go until:

```
inc[1]
```

The decreasing section ending at **r** starts from:


```
dec[r]
```

So we have:

```
Increasing part:
l ----- inc[l]

Decreasing part:
      dec[r] ----- r
```

For the two parts to form a mountain, they must overlap or touch:

```
inc[l] >= dec[r]
```

Therefore:

```
if inc[l] >= dec[r]:
    True
else:
    False
```

20. Query Dry Run

Consider:

```
arr = [2, 3, 4, 5, 4, 3, 2]
       0 1 2 3 4 5 6
```

We calculated:

```
inc = [3, 3, 3, 3, 4, 5, 6]
dec = [0, 1, 2, 3, 3, 3, 3]
```

Query [0, 6]

Increasing part from 0:

```
inc[0] = 3
```

So:

```
0 → 1 → 2 → 3
```

is non-decreasing.

Decreasing part ending at 6:

```
dec[6] = 3
```

So:

```
3 → 4 → 5 → 6
```

is non-increasing.

They meet at:

```
3
```

Check:

```
inc[0] >= dec[6]
```

```
3 >= 3
```

Therefore:

```
true
```

Query [1, 6]

```
inc[1] = 3
```

```
dec[6] = 3
```

Check:

```
3 >= 3
```

Therefore:

```
true
```

The mountain is:

```
3 → 4 → 5 → 4 → 3 → 2  
      PEAK
```

Query [0, 3]

```
inc[0] = 3  
dec[3] = 3
```

Check:

```
3 >= 3
```

True.

The subarray is:

```
[2, 3, 4, 5]
```

Entirely non-decreasing.

Still valid.

Query [3, 6]

```
inc[3] = 3  
dec[6] = 3
```

Check:

```
3 >= 3
```

True.

The subarray is:

```
[5, 4, 3, 2]
```

Entirely non-increasing.

Still valid.

21. Invalid Query Example

Consider:

```
arr = [3, 2, 4]  
      0 1 2
```

Calculate `inc[]`.

At index 0:

```
3 <= 2 ❌
```

Therefore:

```
inc[0] = 0
```

Calculate `dec[]`.

For index 2:

```
arr[1] >= arr[2]
2 >= 4 ❌
```

Therefore:

```
dec[2] = 2
```

Now query:

```
[0, 2]
```

Check:

```
inc[0] >= dec[2]
0 >= 2
```

False.

Therefore:

```
[3, 2, 4]
```

is not a mountain.

22. Complete Code

```
def mountain_queries(arr, queries):
    n = len(arr)

    # -----
    # inc[i] =
    # farthest index reachable from i
    # while the array is non-decreasing
    # -----

    inc = [0] * n
```

```

inc[n - 1] = n - 1

for i in range(n - 2, -1, -1):

    if arr[i] <= arr[i + 1]:
        inc[i] = inc[i + 1]

    else:
        inc[i] = i

# -----
# dec[i] =
# farthest left index from which we can reach i
# while the array is non-increasing
# -----

dec = [0] * n

dec[0] = 0

for i in range(1, n):

    if arr[i - 1] >= arr[i]:
        dec[i] = dec[i - 1]

    else:
        dec[i] = i

# -----
# Answer queries
# -----

ans = []

for l, r in queries:

    if inc[l] >= dec[r]:
        ans.append(True)

    else:
        ans.append(False)

return ans

```

23. Complete Example

```

arr = [2, 3, 2, 4, 4, 6, 3, 2]

queries = [

```

```
[0, 2],  
[1, 3]  
]
```

Array:

```
index: 0 1 2 3 4 5 6 7  
arr:   2 3 2 4 4 6 3 2
```

Build `inc[]`

```
inc = [1, 1, 5, 5, 5, 5, 6, 7]
```

Interpretation:

```
inc[0] = 1
```

because:

```
2 <= 3
```

but:

```
3 <= 2 ❌
```

Build `dec[]`

```
dec = [0, 1, 2, 3, 3, 5, 6, 6]
```

Interpretation:

```
dec[6] = 6
```

because:

```
3 >= 2
```

and the sequence ending at index 6 starts at index 6.

For index 7:

```
3 >= 2
```

Therefore:

```
dec[7] = 6
```

Query [0, 2]

```
inc[0] = 1  
dec[2] = 2
```

Check:

```
1 >= 2
```

False?

But the subarray is:

```
[2, 3, 2]
```

which is clearly a mountain.

This reveals an important issue in the interpretation of the arrays.

The cleanest implementation should instead use:


```
upEnd[i] = farthest right index reachable from i
           while non-decreasing

downStart[i] = farthest left index reachable from i
              while non-increasing
```

Then for query `[1, r]`:

```
upEnd[1] >= downStart[r]
```

For:

```
[2, 3, 2]
```

we have:

```
upEnd[0] = 1
downStart[2] = 1
```

Therefore:

```
1 >= 1
```

True.

24. Correct Interpretation of the Two Arrays

Use these definitions:

`upEnd[i]`

```
upEnd[i] =
the farthest index to the RIGHT reachable from i
while:

arr[i] <= arr[i+1] <= ...
```

Example:

```
[2, 3, 4, 5, 4]
```

Then:

```
upEnd[0] = 3
```

because:

```
2 <= 3 <= 4 <= 5
```

downStart[i]

```
downStart[i] =  
the farthest index to the LEFT reachable from i  
while:  
  
arr[start] >= ... >= arr[i]
```

Example:

```
[2, 3, 4, 5, 4, 3, 2]
```

Then:

```
downStart[6] = 3
```

because:

```
5 >= 4 >= 3 >= 2
```

25. The Most Important Query Formula

For query:

```
[l, r]
```

we need:

```
upEnd[l] >= downStart[r]
```

Why?

Because:

Increasing part:

```
l ----- upEnd[l]
```

Decreasing part:

```
downStart[r] ----- r
```

If:

```
upEnd[l] >= downStart[r]
```

then the two parts overlap.

Therefore, a peak exists.

26. Correct Code Using These Definitions

```
def is_mountain_queries(arr, queries):
    n = len(arr)

    # upEnd[i]:
    # farthest right index reachable from i
    # while non-decreasing

    upEnd = [0] * n

    upEnd[n - 1] = n - 1

    for i in range(n - 2, -1, -1):
        if arr[i] <= arr[i + 1]:
            upEnd[i] = upEnd[i + 1]
```

```
        else:
            upEnd[i] = i

# downStart[i]:
# farthest left index reachable from i
# while non-increasing

downStart = [0] * n

downStart[0] = 0

for i in range(1, n):

    if arr[i - 1] >= arr[i]:
        downStart[i] = downStart[i - 1]

    else:
        downStart[i] = i

ans = []

for l, r in queries:

    if upEnd[l] >= downStart[r]:
        ans.append(True)
    else:
        ans.append(False)

return ans
```

27. Important DP Direction Pattern

This problem teaches a very important general rule.

Suppose:

```
dp[i] depends on dp[i + 1]
```

Then calculate:

```
RIGHT → LEFT
```

Example:

```
inc[i] = inc[i + 1]
```

Therefore:

```
for i in range(n - 2, -1, -1):
```

Suppose:

```
dp[i] depends on dp[i - 1]
```

Then calculate:

```
LEFT → RIGHT
```

Example:

```
dec[i] = dec[i - 1]
```

Therefore:

```
for i in range(1, n):
```

28. The General Rule

Do not decide loop direction based only on the English meaning.

Instead ask:

What previous state does my current state depend on?

If:

```
current = future
```

then:

RIGHT → LEFT

If:

current = previous

then:

LEFT → RIGHT

Table:

Recurrence	Direction
$dp[i] = dp[i - 1]$	Left → Right
$dp[i] = dp[i + 1]$	Right → Left

This appears everywhere in:

- Dynamic Programming
- Prefix/suffix preprocessing
- Reachability arrays
- Monotonic runs
- Next greater/smaller preprocessing
- Suffix maximum/minimum arrays

29. Pattern Name

This problem belongs to the pattern:

Range Query Preprocessing on Monotonic Runs

It also uses:

Directional DP

and:

Left/Right Reachability Preprocessing

The general structure is:

```
Precompute local property
      ↓
Build reachability information
      ↓
Answer each range query in  $O(1)$ 
```

30. Pattern Recognition Checklist

When you see:

```
Given an array
Given many  $[l, r]$  queries
Ask whether each subarray satisfies some property
```

ask:

Question 1

Can the property be represented as a continuous run?

Examples:

```
non-decreasing
non-increasing
same parity
same color
same difference
```

If yes:

```
Precompute run boundaries.
```

Question 2

Can I calculate:

```
farthest reachable index from  $i$ 
```

If yes:

Build a reachability array.

Question 3

Does the current state depend on:

$i - 1$

Then:

left $\rightarrow$ right

Does it depend on:

$i + 1$

Then:

right $\rightarrow$ left

Question 4

Can the query be answered using two boundaries?

For this problem:

increasing range from l

and:

decreasing range ending at r

If they overlap:

valid

31. General Template

```
n = len(arr)

leftReach = [0] * n

# If current state depends on i + 1:
for i in range(n - 2, -1, -1):

    if condition_with_next_element:

        leftReach[i] = leftReach[i + 1]

    else:

        leftReach[i] = i

rightReach = [0] * n

# If current state depends on i - 1:
for i in range(1, n):

    if condition_with_previous_element:

        rightReach[i] = rightReach[i - 1]

    else:

        rightReach[i] = i

for l, r in queries:

    # Compare the two reachable boundaries

    if leftReach[l] >= rightReach[r]:

        answer = True

    else:

        answer = False
```

32. Similar Problems

This pattern is useful for problems involving:

1. Non-decreasing subarray queries

Question:

```
Is arr[l...r] non-decreasing?
```

Precompute:

```
end of increasing run
```

Then:

```
endIncreasing[l] >= r
```

2. Non-increasing subarray queries

Question:

```
Is arr[l...r] non-increasing?
```

Precompute:

```
start of decreasing run
```

Then:

```
startDecreasing[r] <= l
```

3. Good Days to Rob the Bank

The condition is:

```
previous time days are non-increasing
```

and:

```
next time days are non-decreasing
```

This is also:

```
LEFT CONDITION + RIGHT CONDITION
```

4. Longest Bitonic Subarray

A bitonic array is:

```
increasing → decreasing
```

The same mountain idea appears.

Instead of answering queries, we maximize:

```
length
```

5. Longest Mountain in Array

Again:

```
increasing → decreasing
```

Usually solved with:

```
up[] + down[]
```

or two-pointer traversal.

6. Range queries with sorted segments

If a query asks:

```
Is this range sorted?
```

you can precompute:

```
last position where order was violated
```

Then answer each query in:

```
 $O(1)$ 
```

33. Real-World Analogy

Imagine a road.

A mountain road has:

```
Start → climb → peak → descend → end
```

For every query asking:

```
From point l to point r,  
is the road a mountain?
```

we precompute:

```
From each point:  
How far can I continue climbing?
```

and:

```
For each point:  
How far back does the current descending road extend?
```

Then we check:

Does the climbing route and descending route meet?

If yes:

Mountain.

34. Interview Quick Revision Sheet

Problem Pattern

Many range queries
+
Monotonic property

Think:

Precompute monotonic runs.

Mountain Definition

NON-DECREASING → NON-INCREASING

Allowed:

UP
UP → DOWN
DOWN

Not allowed:

DOWN → UP

Increasing Array

```
upEnd[i]
```

Meaning:

```
Farthest right index reachable from i  
while arr is non-decreasing.
```

Condition:

```
arr[i] <= arr[i + 1]
```

Recurrence:

```
upEnd[i] = upEnd[i + 1]
```

Direction:

```
RIGHT → LEFT
```

Decreasing Array

```
downStart[i]
```

Meaning:

```
Farthest left index reachable from i  
while arr is non-increasing.
```

Condition:

```
arr[i - 1] >= arr[i]
```

Recurrence:

```
downStart[i] = downStart[i - 1]
```

Direction:

LEFT → RIGHT

Query

For:

```
[1, r]
```

check:

```
upEnd[1] >= downStart[r]
```

If true:

Mountain

Otherwise:

Not Mountain

Complexity

Preprocessing:

```
O(n)
```

Each query:

```
O(1)
```

Total:

 $O(n + q)$

Space:

 $O(n)$

35. The Most Important Mental Model

Do not memorize:

```
for i in range(n - 2, -1, -1)
```

Instead memorize the dependency:

```
inc[i] depends on inc[i + 1]
```

Therefore:

```
calculate i + 1 first
```

Therefore:

```
RIGHT → LEFT
```

Similarly:

```
dec[i] depends on dec[i - 1]
```

Therefore:

```
calculate i - 1 first
```


Therefore:

LEFT → RIGHT

The ultimate rule is:

The loop direction is determined by the dependency, not by the name of the array.

Final Pattern

```
Many range queries
    ↓
Property is based on continuous monotonic segments
    ↓
Precompute farthest reachable boundaries
    ↓
For each [l, r], compare the boundaries
    ↓
O(n + q)
```

For mountain queries:

```
Increasing from l
    +
Decreasing toward r
    ↓
Do they overlap?
    ↓
Yes → Mountain
No  → Not Mountain
```

This is the core idea to remember.